HEX AWARE · MAVERICKS SEGUE 2026

JAVA TRACK — DAY 5 ASSIGNMENT

Databases, Normalisation and Data Definition

MySQL 8.4 LTS | Complete worked solutions

Prepared by: Haresh Bharadwaj R

Registration No.: 192321037

Assignment date: 11 November 2026

Three application scenarios · 3NF / BCNF · ER models · SQL scripts · CRUD · DDL · Data retrieval

How to use this submission

This solution document explains the design decisions and provides runnable SQL scripts. The three companion .sql files contain the schema, seed data and task demonstrations. Run each script in MySQL 8.4 using MySQL Workbench or the mysql command-line client. Review destructive statements before executing them.

Important: MySQL CHECK constraints are enforced in modern MySQL 8.x. Row counts in information_schema.TABLES can be estimates for InnoDB; use SELECT COUNT(*) for an exact count.

Core database concepts

Concept	Explanation
Database	An organised collection of related data managed by a DBMS.
Primary key	Column(s) that uniquely identify each row; cannot be NULL.
Foreign key	Enforces a relationship by requiring a referenced parent row.
1NF	Atomic values; no repeating groups or multi-valued columns.
2NF	1NF plus every non-key attribute fully depends on the whole candidate key.
3NF	2NF plus no transitive dependency of non-key attributes on a candidate key.
BCNF	For every non-trivial functional dependency X → Y, X is a superkey.

Normalisation reduces duplicated facts and prevents insert, update and delete anomalies. It does not mean splitting every column into a separate table: entities and their functional dependencies determine the decomposition.

Set 1 — Retail Inventory Management

1. Normalisation to 3NF and anomalies

Original columns: SKU, product name, category, supplier name, supplier phone, warehouse, quantity and reorder level. A single row combines product, supplier and warehouse facts. A product may be stocked in multiple warehouses, so the natural inventory key is (SKU, warehouse).

Relation	Attributes / key	Reason
SUPPLIER	supplier_id PK, supplier_name, supplier_phone	Supplier phone belongs to supplier, not each stock row.
CATEGORY	category_id PK, category_name UNIQUE	Category name is stored once.
WAREHOUSE	warehouse_id PK, warehouse_name UNIQUE	Warehouse identity is independent.
PRODUCT	SKU PK, product_name, category_id FK, supplier_id FK, reorder_level	Product-level attributes depend on SKU.
INVENTORY	(SKU, warehouse_id) PK, quantity	Quantity depends on the product–warehouse combination.

Anomaly	Example in the original sheet
Insert	Cannot record a new supplier until a product is stocked.
Update	Changing a supplier phone requires updating every product row; missed rows cause inconsistency.
Delete	Deleting the last SKU from a supplier may also erase the only copy of the supplier's contact details.

2. ER diagram

```
SUPPLIER (1) ■■■■■< (M) PRODUCT (M) >■■■■■ (1) CATEGORY
                         |
                         | (1)
                         v
                   (M) INVENTORY (M) >■■■■■ (1) WAREHOUSE

PRODUCT primary key: SKU
INVENTORY composite primary key: (SKU, warehouse_id)
Foreign keys: PRODUCT.supplier_id, PRODUCT.category_id;
              INVENTORY.SKU, INVENTORY.warehouse_id
```

3. Constraints and sample data

The companion script creates all five InnoDB tables. It uses primary and foreign keys, UNIQUE constraints, mandatory NOT NULL columns, DEFAULT values and CHECK(quantity >= 0). Foreign keys use ON DELETE RESTRICT to protect referenced suppliers/categories/products/warehouses and ON UPDATE CASCADE so a changed parent key propagates.

At least ten rows are inserted into each entity table (ten inventory rows, ten products, ten suppliers, ten categories and ten warehouses). The four commented invalid statements demonstrate duplicate primary key, missing parent, negative quantity and NULL in a mandatory field. Uncomment and run one at a time to capture error evidence.

4. UPDATE and DELETE with before/after SELECT

```
SELECT supplier_id,supplier_name,supplier_phone
FROM suppliers WHERE supplier_id=1;
UPDATE suppliers SET supplier_phone='9888800001'
WHERE supplier_id=1;
SELECT supplier_id,supplier_name,supplier_phone
FROM suppliers WHERE supplier_id=1;

SELECT * FROM products WHERE sku='SKU010';
DELETE FROM inventory WHERE sku='SKU010';
DELETE FROM products WHERE sku='SKU010';
SELECT * FROM products WHERE sku='SKU010';
```

Delete the dependent inventory row first because the foreign key restricts deleting a product that is still referenced. The final SELECT returns no product row.

5. MySQL system databases and table metadata

System schema	Purpose
information_schema	Read-only metadata about databases, tables, columns, constraints and privileges.
mysql	Accounts, authentication, roles and privilege data.
performance_schema	Runtime instrumentation for statements, waits, stages and server activity.
sys	Convenience views built over Performance Schema and Information Schema.

```
SELECT TABLE_NAME, ENGINE, TABLE_ROWS
FROM information_schema.TABLES
WHERE TABLE_SCHEMA='retail_inventory'
  AND TABLE_TYPE='BASE TABLE';
```

TABLE_ROWS for InnoDB is an estimate. For exact row counts, query each table with COUNT(*).

Set 2 — Bank Branch Accounts

1. 3NF / BCNF decomposition and keys

A bank account belongs to one customer, one account type, one branch and one relationship manager. Customer address depends on customer_id; branch name depends on branch_id; account type name depends on account_type_id; manager contact depends on manager_id. These facts should not be repeated on every account row.

Relation	Attributes / key
CUSTOMER	customer_id PK, customer_name, address_line

Relation	Attributes / key
BRANCH	branch_id PK, branch_name UNIQUE, branch_code UNIQUE
ACCOUNT_TYPE	account_type_id PK, type_name UNIQUE
RELATIONSHIP_MANAGER	manager_id PK, manager_name, email UNIQUE
ACCOUNT	account_id PK, account_number UNIQUE, customer_id FK, account_type_id FK, branch_id FK, manager_id FK, opened_on, balance

Candidate keys: in ACCOUNT, account_id and account_number are candidate keys (each uniquely identifies an account); account_id is selected as the primary key and account_number is an alternate key. The composite key (customer_id, account_type_id) is not guaranteed unique because a customer may hold multiple accounts of the same type. A composite key is not required in the chosen account model. The natural candidate key of BRANCH is branch_code (branch_name is also declared unique).

2. ER diagram

```
CUSTOMER (1) ■■■■■< ACCOUNT >■■■■■ (1) ACCOUNT_TYPE
                        |
BRANCH (1) ■■■■■■■■■■■■■■+
                        |
RELATIONSHIP_MANAGER (1) ■

ACCOUNT has account_id PK and account_number UNIQUE.
Each account references exactly one customer, type, branch and manager.
```

3. Data types and constraints

Field	Type	Justification
IDs	INT / BIGINT AUTO_INCREMENT	Surrogate numeric keys; BIGINT for account volume if required.
Account number	VARCHAR(20) UNIQUE	Preserves leading zeros and possible letters.
Money	DECIMAL(15,2)	Exact fixed-point decimal; FLOAT/DOUBLE can introduce rounding.
Opened date	DATE	Stores calendar date without a time component.
Names / address	VARCHAR	Variable-length human-readable text.
Account type	VARCHAR with CHECK / lookup table	Restricts allowed values; lookup table supports maintainability.

The script uses CHECK(balance >= 0), a default balance of 0.00, AUTO_INCREMENT surrogate keys and UNIQUE account numbers. The account type table restricts values to SAVINGS, CURRENT, SALARY and FIXED_DEPOSIT.

4. DELETE vs TRUNCATE vs DROP

	Effect	AUTO_INCREMENT / rollback
...	Removes selected rows; WHERE can limit the scope.	Usually does not reset the counter. InnoDB DELETE can be rolled back inside an explicit t
	Removes all rows by deallocating table data pages; no WHERE clause.	Resets AUTO_INCREMENT. DDL with implicit commit; not normally rollbackable.
	Removes table definition and data.	Table no longer exists; DDL implicit commit and not rollbackable.

The companion script creates ddl_test and runs each operation. The expected next ID after deleting id=1 from a table that had ids 1 and 2 is normally 3. TRUNCATE resets the counter, so the next inserted row normally receives 1. DROP removes the table entirely. Exact behaviour should be verified on the configured MySQL 8.4 instance.

5. Retrieval using logical, membership, range and pattern operators

```
SELECT a.account_number,c.customer_name,b.branch_name,
       t.type_name,a.opened_on,a.balance
FROM accounts a
```

```
JOIN customers c ON a.customer_id=c.customer_id
JOIN branches b ON a.branch_id=b.branch_id
JOIN account_types t ON a.account_type_id=t.account_type_id
WHERE a.opened_on BETWEEN '2026-10-01' AND '2026-12-31'
  AND a.balance > 20000
  AND (t.type_name IN ('SAVINGS','SALARY')
       OR c.customer_name LIKE 'A%');
```

BETWEEN selects the inclusive date range; AND requires both date and balance conditions; IN checks a list of account types; OR permits either an allowed type or a customer name beginning with A; LIKE performs pattern matching.

6. ALTER TABLE exercises

```
ALTER TABLE accounts
ADD COLUMN kyc_status ENUM('PENDING','VERIFIED','REJECTED')
NOT NULL DEFAULT 'PENDING';

RENAME TABLE relationship_managers TO account_managers;

-- First inspect the actual CHECK constraint name:
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA='bank_branch' AND TABLE_NAME='accounts';
-- Then substitute the returned name:
ALTER TABLE accounts DROP CHECK accounts_chk_1;
```

Adding the column assigns PENDING to existing rows through the DEFAULT. RENAME TABLE changes the table name without changing its rows. DROP CHECK removes the selected check rule; inspect the generated constraint name first rather than assuming it is always accounts_chk_1.

Set 3 — Hospital Patient Records

1. Normalisation to 3NF

The original wide sheet repeats patient, doctor and department details for every visit. The decomposition separates the five required entities.

ation	Attributes / key	Dependency
IENT	patient_id PK, patient_name, address_line, phone	Patient details depend on patient_id.
PARTMENT	department_id PK, department_name UNIQUE	Department name depends on department_id.
CTOR	doctor_id PK, doctor_name, department_id FK	Doctor belongs to a department.
T	visit_id PK, patient_id FK, doctor_id FK, visit_date, diagnosis, prescription, status	Visit facts depend on visit_id.
LING	billing_id PK, visit_id FK UNIQUE, bill_amount, paid_amount	Billing details depend on billing_id; one billing record per visit in this mo

Step 1: move patient attributes to PATIENT so address is not repeated for each visit. Step 2: move department facts to DEPARTMENT and reference it from DOCTOR. Step 3: move doctor facts to DOCTOR and reference the doctor from VISIT. Step 4: keep visit-specific date, diagnosis and prescription in VISIT. Step 5: move bill amounts to BILLING and link them to the visit. This removes transitive dependencies and repeated descriptive data.

2. ER diagram

```
PATIENT (1) ■■■■■< (M) VISIT (M) >■■■■■ (1) DOCTOR
                                            |
                                            | (M:1)
                                            v
                                        DEPARTMENT

VISIT (1) ■■■■■■■■■ (0..1) BILLING

Each visit belongs to one patient and one doctor.
Each doctor belongs to one department.
Each visit may have one billing record.
```

3. Constraints and sample data

The schema uses DATE for visit_date, CHECK(visit_date <= CURRENT_DATE), DECIMAL(12,2) for bill amounts, CHECK constraints to prevent negative amounts and a status default of OPEN. The billing table also checks paid_amount <= bill_amount. Ten rows are inserted into each of the five tables.

A future visit date is rejected by the CHECK rule. Bill amounts cannot be negative. The status is automatically OPEN when omitted from an INSERT.

4. Database and table administration

```
CREATE DATABASE IF NOT EXISTS hospital_training;
USE hospital_training;
CREATE TABLE demo_table(id INT PRIMARY KEY, note VARCHAR(40));
RENAME TABLE demo_table TO renamed_demo_table;
DROP DATABASE hospital_training;
CREATE DATABASE hospital_training;
```

CREATE DATABASE creates a schema. RENAME TABLE changes the table's name. DROP DATABASE is destructive: it removes the database and its tables/data from the server's storage. Recreating the database creates an empty schema; it does not restore dropped records. Back up before running DROP DATABASE.

5. CRUD and retrieval

```
SELECT * FROM visits WHERE visit_id=1;
UPDATE visits SET status='CLOSED' WHERE visit_id=1;
DELETE FROM billing WHERE visit_id=10;
DELETE FROM visits WHERE visit_id=10;

SELECT DISTINCT p.patient_id,p.patient_name,v.visit_date
FROM patients p JOIN visits v ON p.patient_id=v.patient_id
WHERE v.visit_date >= '2026-10-01'
  AND v.visit_date < '2026-11-01'
ORDER BY v.visit_date;

SELECT d.doctor_name,dep.department_name
FROM doctors d
JOIN departments dep ON d.department_id=dep.department_id
ORDER BY dep.department_name,d.doctor_name;
```

The date range is half-open: it includes every date in October 2026 without relying on a last-day timestamp. Delete the billing row before deleting the visit because of the foreign key.

6. SQL vs NoSQL

QL (relational)	NoSQL (document example)
ables, rows, columns and defined relationships.	Flexible documents/collections; fields may vary.
CID transactions and constraints are strong fit for linked clinical and billing records.	Consistency and transaction support depend on the database; many systems supp
ommonly scales vertically; read replicas and sharding are also available.	Often designed for horizontal distribution; actual capability depends on product and
tructured patient, doctor, visit and billing records with joins and integrity constraints.	Flexible clinical notes, event payloads, rapidly evolving or nested data.

For this hospital example, a relational database is a strong default because visits, patients, doctors and billing have clear relationships and integrity requirements. NoSQL can complement it for flexible documents or high-volume event data. The comparison is general; SQL and NoSQL each include products with different consistency and scaling models.

Execution checklist and evidence

Evidence item	How to produce it
Schema creation	Run each Set SQL script and capture successful CREATE TABLE results.
Seed data	Run INSERT sections; verify each table with SELECT COUNT(*).
ER diagrams	Use the relationship diagrams in this PDF; recreate in MySQL Workbench if a graphical export is required.

Evidence item	How to produce it
Failing inserts (Set 1)	Uncomment each invalid INSERT separately and capture the MySQL error.
Before/after DML	Run SELECT, UPDATE/DELETE, then SELECT again and capture both outputs.
DDL behaviour (Set 2)	Run the test table sequence and record the generated IDs after DELETE and TRUNCATE.
System metadata	Run the information_schema query; note InnoDB row counts may be estimates.

The SQL files are executable examples, but the evidence screenshots must be captured from an actual MySQL 8.4 run. They are not fabricated or represented as already executed.

Submission files: Set1_Retail_Inventory.sql · Set2_Bank_Branch_Accounts.sql · Set3_Hospital_Patient_Records.sql · this solution PDF.